3D Attention U-Net for CT Scan Segmentation

Andy F.

Department of Electrical and Systems Engineering
Washington University in St. Louis



May 2025

Project Overview

Introduction and Background

Medical image segmentation is critical for identifying anatomical structures in CT, MRI, and PET scans, supporting diagnosis, treatment planning, and disease assessment. Manual segmentation is time-consuming and inconsistent, particularly for complex datasets requiring precise labeling of multiple regions (spine, femurs, pelvis).
Deep learning models, especially U-Net architectures, have become standard in medical imaging. The U-Net employs a symmetric encoder-decoder framework with skip connections that propagate feature channels during upsampling, creating its distinctive U-shape.

U-Net

U-Net processes images through an encoder path that extracts hierarchical features via convolutions and downsampling, followed by a decoder path that restores spatial resolution through upsampling. Skip connections preserve spatial details lost during downsampling. Despite its effectiveness, U-Net struggles with complex images as skip connections may carry irrelevant background information, and the network lacks internal prioritization mechanisms.

Attention U-Net

Attention gates were introduced to address these limitations, automatically suppressing irrelevant activations and highlighting important regions based on context. These gates compute attention coefficients that scale encoder features before they reach the decoder, ensuring only task-relevant information passes through skip connections.

Project Motivation

This project modernizes an existing biomedical segmentation pipeline by transitioning from TensorFlow to PyTorch and upgrading from 2D slice-wise to 3D volumetric segmentation. PyTorch offers greater developer flexibility, cleaner model definition, and better integration with research tools. The 3D approach addresses key limitations of 2D segmentation:

- 2D networks lack context from neighboring slices
- 2D requires artificial positional encoding
- Medical scans are inherently 3D

Implementation

The 3D PyTorch implementation features:

- Direct handling of volumetric HDF5 files instead of 2D TFRecords
- Native 3D spatial context without artificial positional encoding
- Conv3d, MaxPool3d, and ConvTranspose3d operations for volumetric processing
- 3D attention gates that filter volumetric feature maps
- Loss functions adapted for 5D tensors (batch, channels, depth, height, width)
- Voxel-wise evaluation metrics for entire volumes
- Batch size of 1 (full volume) with Adam optimizer
- Volumetric post-processing with 3D connected component analysis

Benefits of 3D approach:
The volumetric implementation provides several advantages:

- Captures inter-slice relationships for better anatomical learning
- Eliminates the need for artificial position information
- Improves prediction continuity across the volume
- Provides more stable metrics through whole-structure evaluation
- Reduces inconsistencies at structure boundaries

Methods

Checker Code

Due to the complicated structure of the project, a checker code file is first created to act as a critical validation tool in this project.
The checker script is a standalone testing code to verify the correctness of the key pipelines without running a full training cycle. It is a sanity check for the 3D attention U-Net implementation by loading a single 3D CT HDF5 scan and its corresponding segmentation mask, preprocessing the data through the same transformation pipeline, running a forward pass throught the model to generate predictions, computing both dice score and the complete loss function, and verifies the

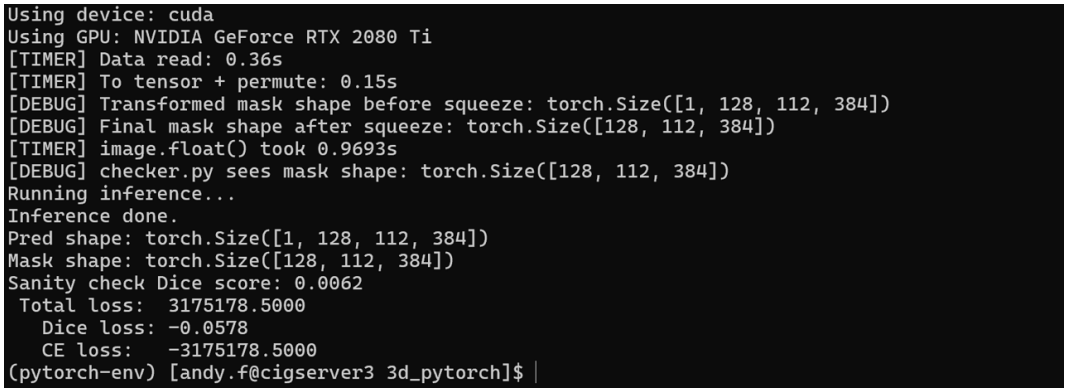


Figure 1: Checker code console output

shape compatibility between inputs, outputs, and ground truth.

Problems did rose during the sanity check: mostly due to one-hot encoding. During the data loading process, time stamps are placed at every step in the dataloader since the code was running for long time even for every small volume (cropped sizes). Therefore, the problem was found out by the time stamps because the code stops at one-hot encoding and not moving forward. Therefore, this contributes to the overall project by providing one of the most important aspect of the project - dataloading. Thus, the one-hot encoding step in every stage of the project is replaced by the label-space mask. Here is the output of the checker code validating the important procedures of the project:

Debugging Process

After running the checker code, some debugging processes were still necessary to ensure proper model operation.

Data Flow Validation was completed through monitoring tensor shapes at each layer. This step ensured that the 3D volumes maintained propery dimensionality through the encoding and decoding paths.

Memory management presented significant challenges when processing the large 3D volumes. Serveral optimizations were attempted: used smaller cropping/padding dimensions (96,96,160) for training; implemented batch size 1; added explicit $torch.cuda.empty_cache()$ calls after each training iteration.

The combined loss functions also required attention during debugging: validation checks were implemented to ensure proper weight normalization, correct device placement of loss weights to match tensor devices (always on GPU).

Mixed Precision Training was used to also manage GPU memory issues:

Mixed precision training is a method that uses both full precision - floating-point 32 format (FP32), and lower precision - FP16 or BF16 floating-point format - during model training to gain performance without sacrificing the model accuracies.
The mixed precision uses lower precision for operations such as during forward and backward passes, and higher precision for operations such as parameter updates and accumulations.
The training process implemeneted with mixed precisino training utilized Pytorch's automatic mixed precision framework:

- *GradScaler* was initialized at the beginning of the training to prevent underflow in gradients during backpropagation - scales up the loss by a large factor to prevent loss of information during computation, then unscales it before the optimizer step.
- *autocast* automatically casts operations to lower precisions, where appropriate.
- *scaler.step*(*optimizer*) and *scaler.update*() were employed to dynamically adjust scaling factors.

After implementing mixed precision training, now the 3D volume can be passed to the full-size model with depth of 5 of the attention U-Net, allowing better training accuracies.

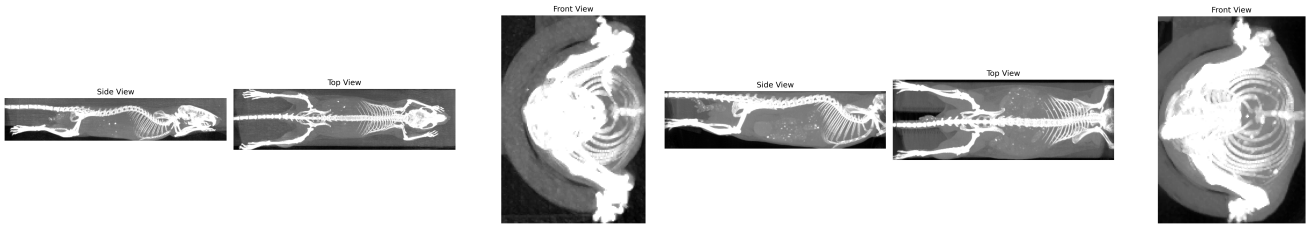
Current Model Parameters

Current model has the following parameters thanks to previous optimization procedures:

- **Model**: 5-depth Attention U-Net with initial feature channel number of 64.
- **3D Volume Input Size**: $(96, 96, 160)$ following D, H, W.
- **Batch Size**: 1
- **Epochs:** 50
- **Training Set**: 3 mice scans
- **Validation Set**: 1 mouse scan
- **Testing Set**: 1 mouse scan

The $MiceCT3DDataset$ class loades the 3D HDF5 volume and corresponding segmentation masks using h5py. Then images are normalized using TochIO's ZNormalization.
Combined loss function of cross-entropy and dice losses are implemented.

$$TotalLoss = (w_{ce} * CE_{loss}) + (w_{dice} * Dice_{loss}) \tag{1}$$



(a) 60 kVp Overview (b) 80 kVp Overview

Figure 2: 3D CT Overview

The Adam optimizer is used during training, and the learning rate is scheduled to start at 0.0007, increases linearly to 0.0008, and then decays exponentially to 0.0003 towards the end of training.

Experiments and Results

Original Dataset

The original CT scan can be visualized using **Maximum Intensity Projection**. It projects highest intensity value along a specific angle onto 2D view. The MIPs are created along three anatomical planes:

- Sagittal (side view) - projection along axis 0
- Coronal (top view) - projection along axis 1
- Axial (front view) - projection along axis 2

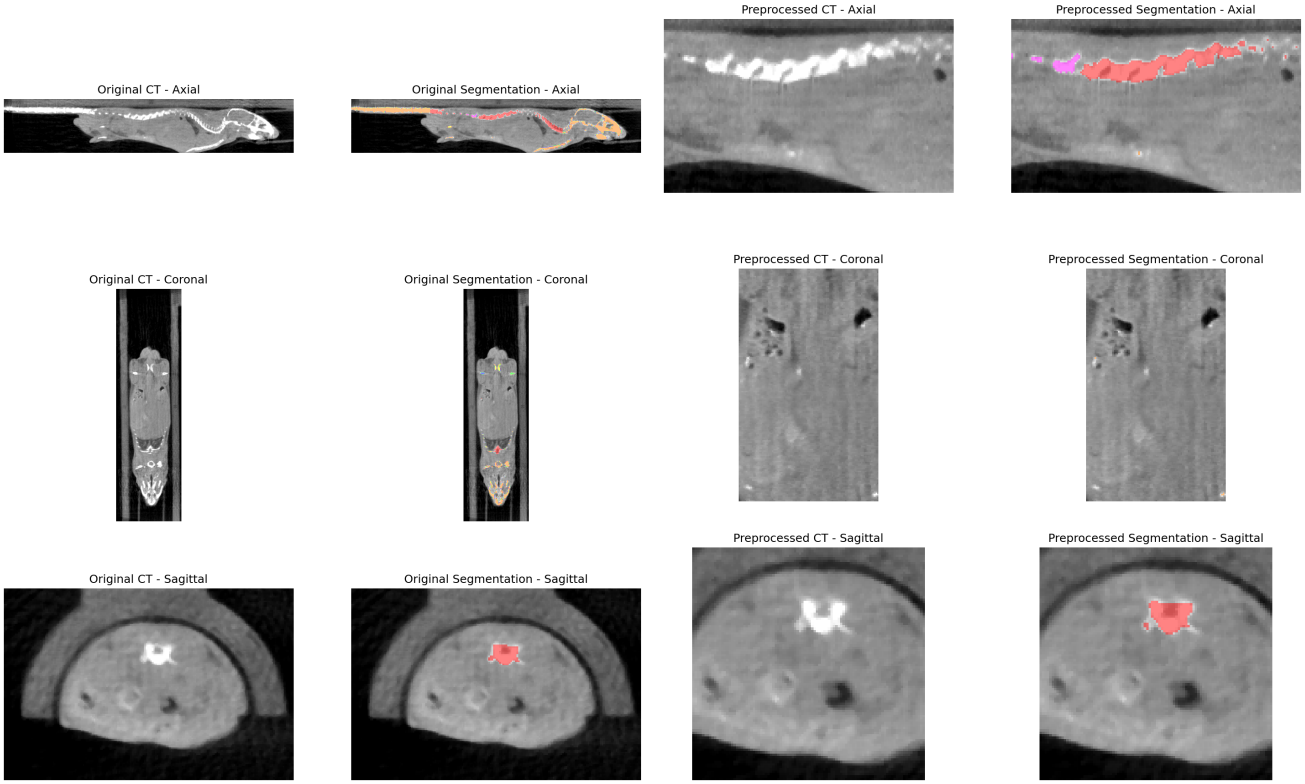
This provides an overview of the 3D structure of the CT scan without going through every individual slices.

Preprocessed Data

Since one full 3D CT scan is too large to be forwarded directly into the model, it is preprocessed before being feed. Cropping are applied to the scan to make the size from around $(184, 122, 657)$ to $(96, 96, 160)$.

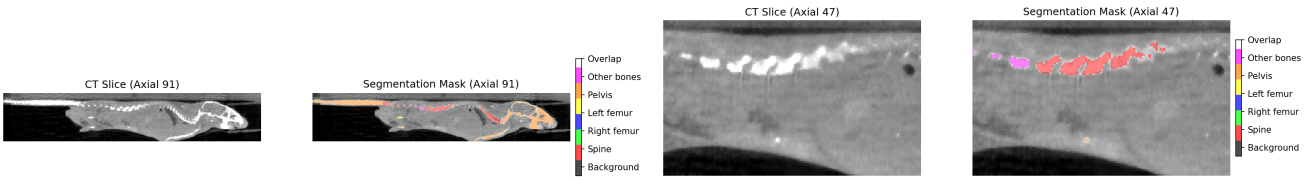
Comparison of the pre-processed and the original 3D overview (fig 3): The axial and coronal dimensions are extensively cropped, but the sagittal view is less cropped compared to the two other dimensions. This might be causing issues such as missing information: from the axial view, the information of pelvis is already missing.

From the comparison between the original and preprocessed representative axial slice (fig 4), it



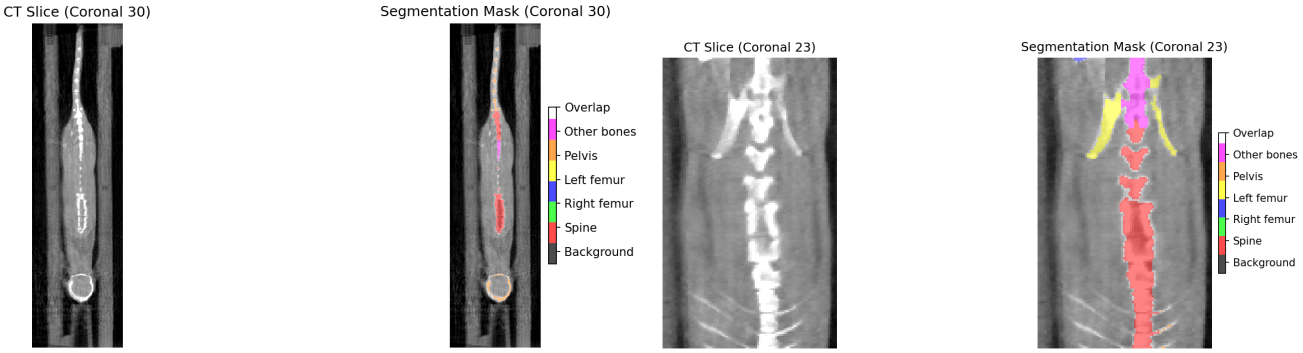
(a) Original 3D Overview (b) Preprocessed 3D Overview

Figure 3: Side by Side Comparision of Original and Preprocessed 3D Overview



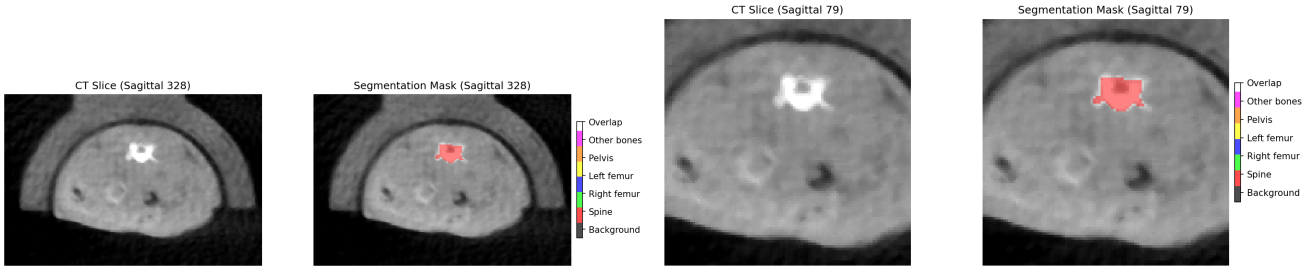
(a) Original representative axial slice (b) Preprocessed representative axial slice

Figure 4: Side by Side Comparison of Original and Preprocessed Representative Axial Slice



(a) Original representative coronal slice (b) Preprocessed representative coronal slice

Figure 5: Side by Side Comparison of Original and Preprocessed Representative coronal Slice



(a) Original representative sagittal slice (b) Preprocessed representative sagittal slice

Figure 6: Side by Side Comparison of Original and Preprocessed Representative Sagittal Slice

can be concluded that some of the spine and other bones in the center of the mouse is preserved, but structures on the two ends of the mouse - head portion and tail portion - are completely cropped.

From the comparison between the original and preprocessed representative coronal slice (Fig. 5), it can be seen that again the structures in the center, such as spines, left femur, and just a slight of pelvis, are included in the cropped version.

From the Comparison of the original and preprocessed representative sagittal slice (Fig. 6): most of the structures are preserved, largely due to its information was less than other two axis in nature.

Training Results

As shown in Fig. 7, during the initial training process, there was high validation loss spike at around epoch 3-4 reaching over 2000, indicating possible instability in early training process. Both training and validation Dice score show initial instability with training Dice spiked to around 0.9 before dropping.

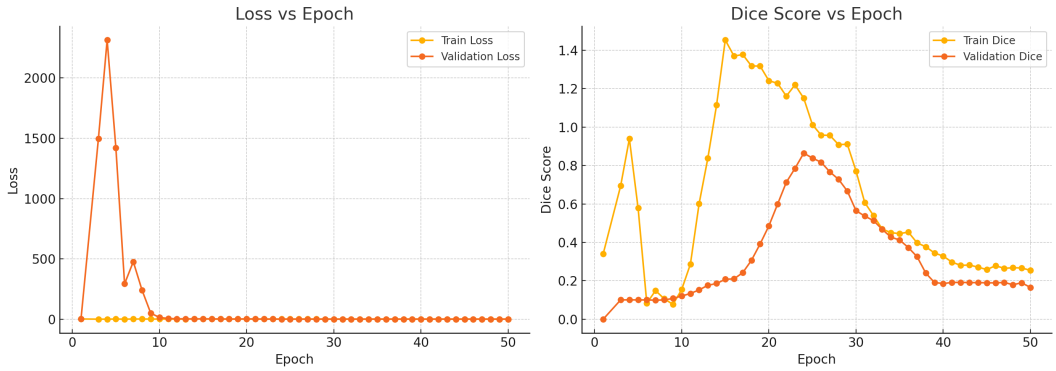


Figure 7: Loss / Dice Score vs. Epoch

During the middle of the training from epoch 10 to 30, the loss values stabilizes significantly for both train and validation. Between epoch 12-25, training Dice scores increased largely to around 1.4, while the validation Dice improves more gradually. The best validation Dice score reaches around 0.85 at epoch 24.

Toward the end of the training phase from epoch 30 to 50, both training and validation Dice scores steadily decreased after epoch 25. The gap between training and validation Dice narrows, both settling around 0.2-0.3 by the end.

Emerging issues from the training process can be inferred:

The model might be overfitting because of the large gap between training and validation Dice. This needs to be further investigated because 50 epochs is very short training process; however, because of the small dataset size with only 3 training datasets, this might be possible. Moreover, both training and validation performance decreases later in the training process, indicating possible optimization issues.

The learning rate might also be causing issues: the learning rate scheduler might be problematic, since after 30 epochs, the model appears to be "underlearning".

Training Dice exceeding 1.0 is unusual; therefore, the loss function needs to be further investigated. Moreover, the balance used for CE and $Dice$ loss can be adjusted to examine the problem.

The extreme loss spike at the beginning of the training process suggests a potential instability.

Prediction

From the overview prediction (Fig. 8) of the trained model previously, it captured major anatomical structures in three planes and recognized the spine correctly in some regions. However, there is extreme over-segmentation of the pelvis (orange area) across all views. The discrimination between classes is very poor. There is also excessive background misclassification. This might be

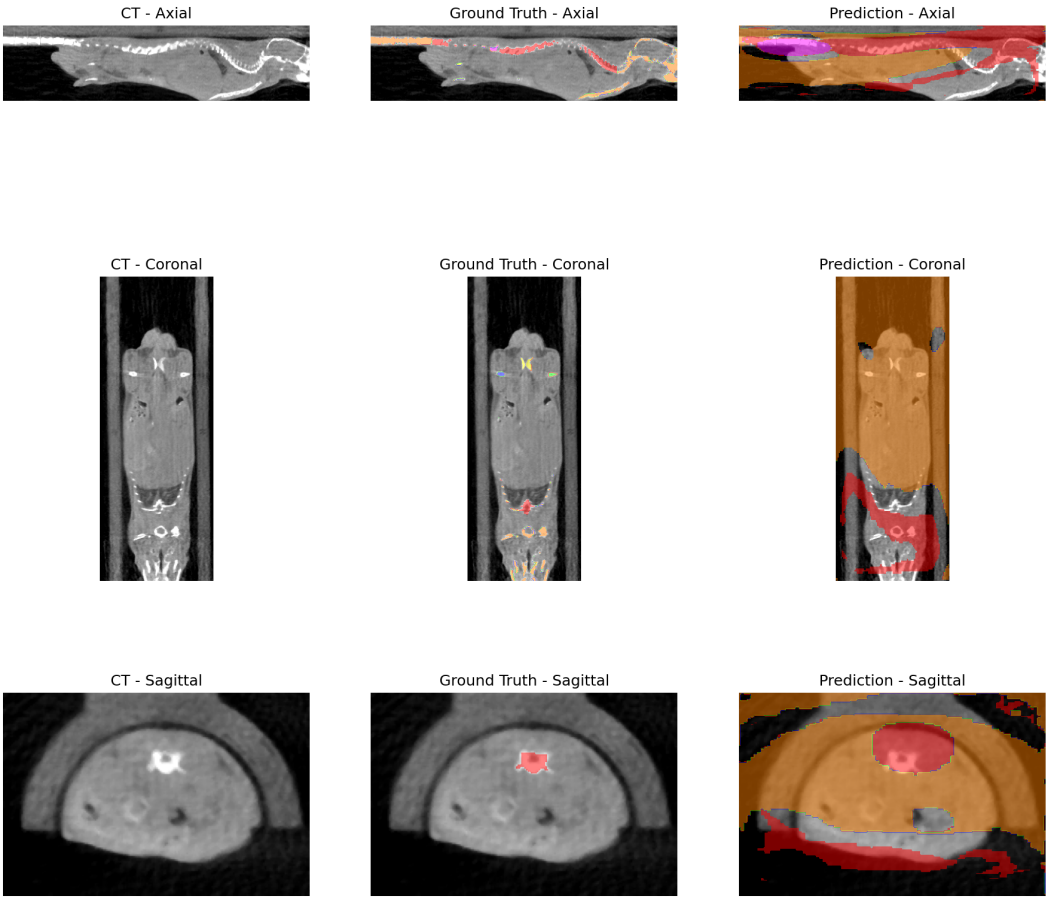


Figure 8: Prediction 3D Overview

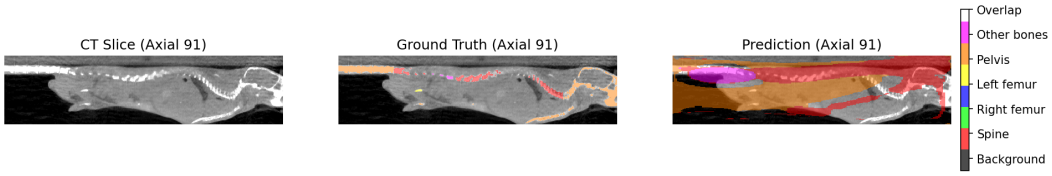


Figure 9: Representative Axial Prediction Slice *no.*91

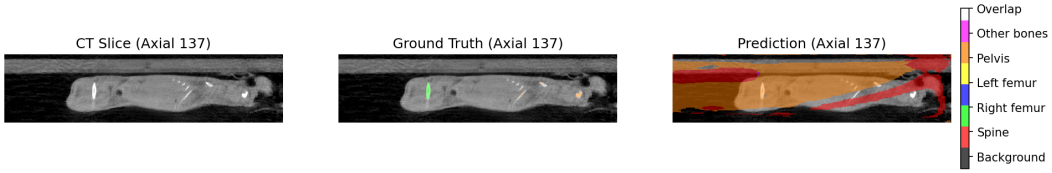


Figure 10: Representative Axial Prediction Slice *no.*137

caused by class imbalance in the training data - pelvis being dominant, which aligns with the problem mentioned earlier in the preprocessed data - pelvis might be the structure that exist the most in the cropped data, as well as lack of training examples of femur positions.

Axial

The spine was partially detected (Fig. 9), and some "other bones" classification appears correct. However, similar over-segmentation of pelvis problem is still present in this slice. The spine segmentation extends incorrectly, and the boundaries are not precise. Possible causes are that the model receptive field may be too large for cropped data, causing context bleeding.

Possibly because of the dominance of spines and pelvis in the training dataset, the prediction in Fig. 10 is largely incorrect.

Coronal

The prediction along the coronal axis (Fig. 11) is similar to the axial prediction of slice number 91 (Fig. 9), which identified the main spine region and some "other bones" area, and massive over-segmentation of pelvis region. Also, there are irregular, noisy boundaries with background. Similar behavior appears in Fig. 12.

Sagittal

The sagittal predictions (Fig. 13) again, follow the similar behavior shown in other axis, partially correct segmentation of spines, over-segmentation of pelvis, and noisy background bound-

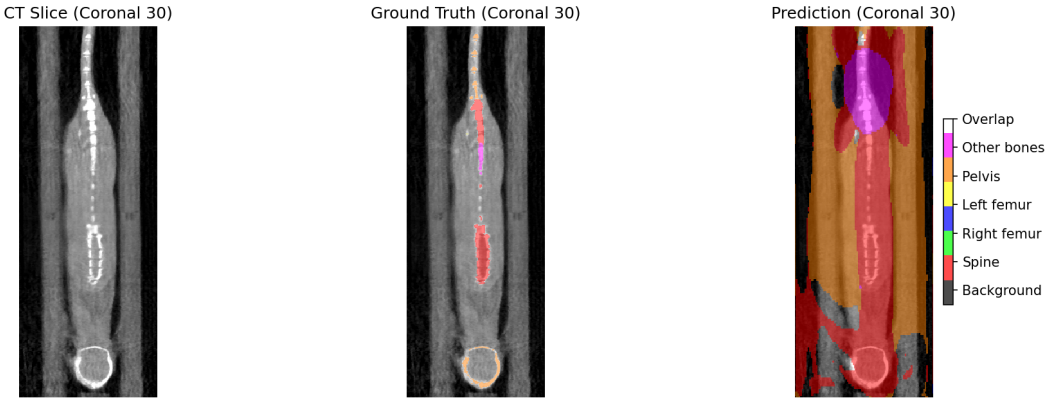


Figure 11: Representative Coronal Slice *no.*30

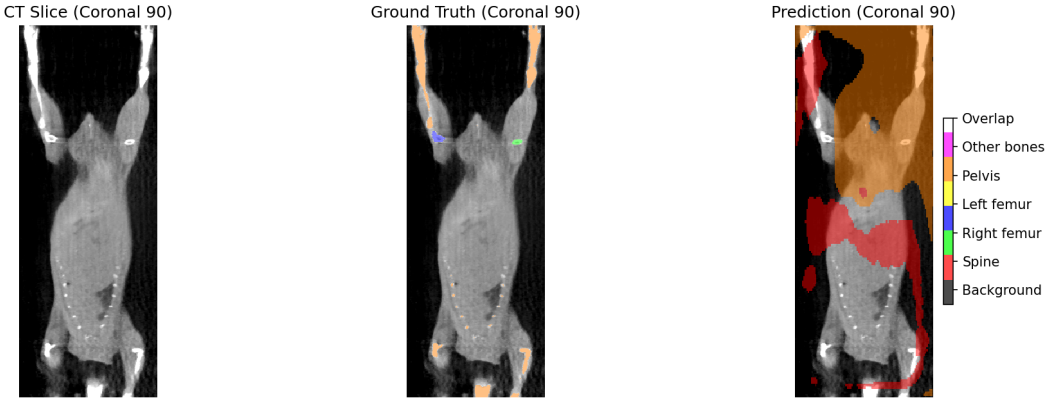
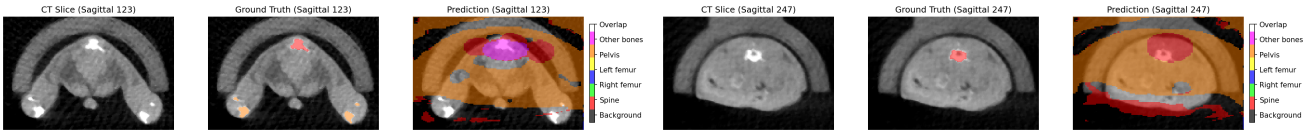


Figure 12: Representative Coronal Slice *no.*90



(a) Representative Sagittal Slice *no.*123 (b) Representative Sagittal Slice *no.*247

Figure 13: Two Representative Sagittal Prediction

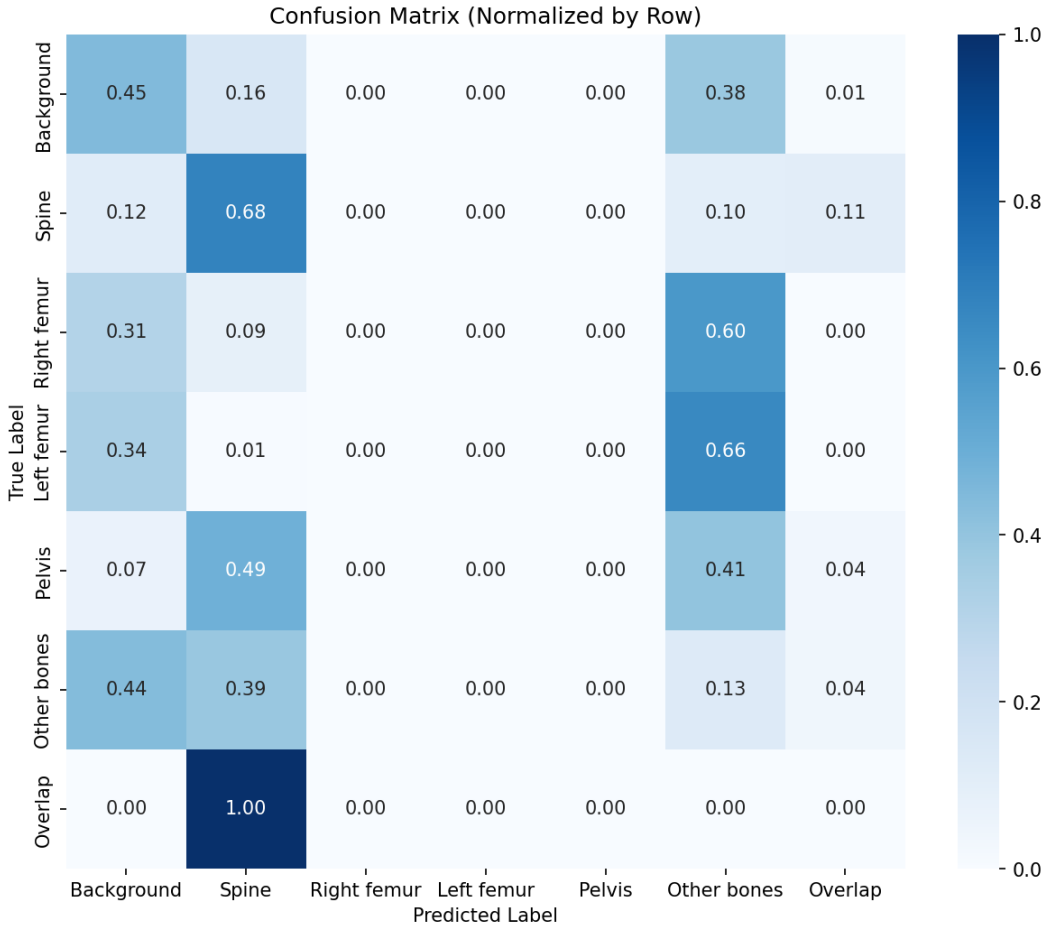


Figure 14: Prediction Confusion Matrix

aries.

Prediction Confusion Matrix

The confusion matrix (Fig. 14) from the prediction provides quantitative insights into the model's performance across different structure classes.

Spine has the strongest detection with 68% of correct classification. 100% of "overlap" regions are incorrectly classified as spine. There are substantial confusion between other classes and spine (39% of "other bones" and 49% of pelvis misclassified as spine).

Even with dominance of pelvis, the classification femur and pelvis are completely incorrect. No voxels were classified as femur.

45% of **background** regions were correctly identified, 38% of background were incorrectly labeled as "other bones".

The confusion matrix reveals that there is a strong class imbalance in training data and small

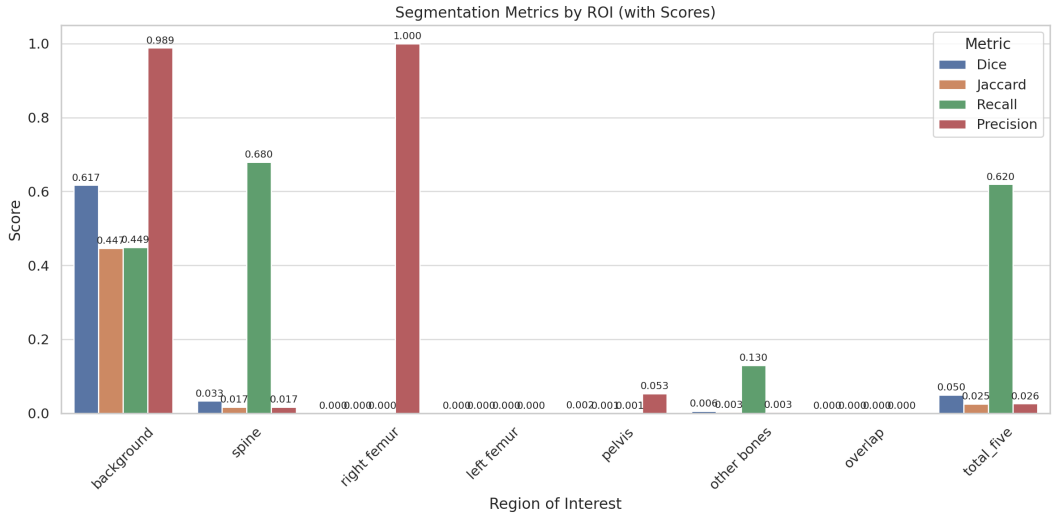


Figure 15: Class Scores Summary

structures such as femurs and pelvis regions were unable to be detected by the model.

Results Summary

The background classification shows reasonable performance with Dice of 0.617 and 0.989 of precision. All bone structures except spine show near-zero performance across most metrics. Despite visual evidence of spine detection in images, quantitative metrics are poor: very low Dice (0.033) and Jaccard (0.017) scores, high recall (0.680) but extremely low precision (0.017) indicates massive over-segmentation - suggests the model is labeling many non-spine regions as spine.

The contrast between recall and precision for spine (0.680 vs 0.017) indicates the model is over-predicting. This reinforces observations from the images showing excessive segmentation.

Causes The model is heavily biased toward background classification, suggesting the loss function may not be properly addressing class imbalance.

The attention mechanism may not be capturing anatomical context properly.

The model appears to prioritize recall over precision for bone structures.

The degradation seen in the training curves coupled with these metrics suggests the model was not optimized effectively.

Further Steps

Training Strategy

To address class imbalance: implement more effective class weighting strategies based on both frequency and physical volume; explore specialized loss functions designed for highly imbalanced segmentation tasks; implement better learning rate scheduler; increase training epochs.

3D Patches

The current implementation's limitation can be substaintially improved by adopting the 3D-patch based approach for training.

Patches of 3D can be divided into manageable sizes to allow deeper networks while staying within GPU memory constrains. It enables larger batch sizes which can stabilize training and improve convergence.

Smaller patches encourage the model to focus on local anatomical patterns and better captures small structures like femurs that were missed in the current approach.

Patch extraction itself serves as an implicit augmentation technique, and overlapping patches create multiple views of boundary regions. Rotation, scaling, and intensity augmentations can be applied more effectively to patches.

By implementing a 3D patch-based approach, it can be expected to have significant improvements in segmentation accuracy, particularly for the femurs and other small bone structures that showed near-zero performance in our current implementation. This approach would better utilize the model's capacity to learn detailed anatomical features while maintaining computational feasibility for high-resolution 3D volumes.